

Desarrollo de un Manual de Despliegue de API's

Guía de arquitectura híbrida (MySQL & MongoDB), variables de entorno, contenedorización con Docker, administración de procesos con PM2 y proxy inverso con Nginx en entornos de alta disponibilidad.

PROYECTO

QUANTIFY — Habit Tracker

AUTOR

Equipo de Ingeniería QUANTIFY

VERSIÓN

v1.0.0 (Producción)

FECHA

Mayo 19, 2026

Índice de Contenido

1. Introducción y Arquitectura Operativa	03
2. Requisitos Previos del Sistema	04
3. Configuración Detallada del Entorno (.env)	04
4. Puesta en Marcha en Desarrollo	05
5. Despliegue Tradicional con PM2 (Alta Disponibilidad)	06
6. Despliegue Containerizado con Docker Compose	07
7. Configuración de Nginx y Certificado SSL (Proxy Inverso)	08
8. Mantenimiento, Copias de Seguridad y Monitoreo	09
9. Guía de Solución de Problemas (Troubleshooting)	10

1. Introducción y Arquitectura

El backend de **QUANTIFY** ha sido diseñado bajo un paradigma híbrido de persistencia de datos. Esta arquitectura combina las fortalezas del almacenamiento relacional y documental, logrando una combinación perfecta entre consistencia ACID y escalabilidad flexible.

Esquema de Bases de Datos Híbrido

A diferencia de sistemas monolíticos que dependen de una sola base de datos, QUANTIFY segmenta sus responsabilidades de la siguiente manera:

● MySQL (Sequelize ORM)

Gestiona los datos estructurados altamente relacionales: cuentas de usuario, control estricto de hábitos, registros diarios de métricas de salud, conquistas de logros e historial transaccional de auditoría (Bitácora).

● MongoDB (Mongoose ODM)

Maneja almacenamiento de documentos semiestructurados como tickets de soporte técnico, retroalimentación del usuario y almacenamiento secundario flexible que no requiere integridad referencial estricta.

📌 Nota sobre Sincronización Automática

En entornos de desarrollo local, el servidor de Express sincroniza automáticamente los modelos de Sequelize llamando a `sequelize.sync({ alter: true })` en su inicialización, además de poblar los usuarios administradores necesarios si no existiesen.

2. Requisitos & 3. Configuración

2. Requisitos Previos del Sistema

Asegúrese de que el entorno cumpla con las siguientes dependencias:

- **Node.js:** Versión mínima `18.x` . Recomendado `20.x LTS` .
- **MySQL Server:** Versión `8.0+` , configurado con codificación `utf8mb4` .
- **MongoDB Instance:** Conexión local activa o una instancia configurada en **MongoDB Atlas**.

3. Variables de Entorno (.env)

El backend utiliza el archivo `.env` ubicado en la raíz de la carpeta `/backend` para su correcto funcionamiento. A continuación se listan y explican las variables utilizadas:

Variable	Valor de Ejemplo	Descripción
<code>PORT</code>	<code>5000</code>	Puerto en el que se ejecuta la API de Express.
<code>MYSQL_HOST</code>	<code>localhost</code>	Dirección IP o host del servidor MySQL.
<code>MYSQL_USER</code>	<code>root</code>	Usuario administrativo de MySQL.
<code>MYSQL_DATABASE</code>	<code>quantify_db</code>	Nombre de la base de datos MySQL relacional.
<code>MONGO_URI</code>	<code>mongodb+srv://...</code>	String de conexión (SRV) para MongoDB.
<code>JWT_SECRET</code>	<code>un_secreto_robusto...</code>	Semilla usada para firmar los tokens de sesión.
<code>MAILTRAP_USER</code>	<code>faee107ba8fa9f</code>	Usuario SMTP para envío de notificaciones por mail.

Seguridad de Producción

NUNCA exponga o suba el archivo `.env` a repositorios de control de versiones Git. Genere siempre una clave segura y robusta de al menos 64 caracteres usando `openssl rand -hex 64` para la variable `JWT_SECRET`

en servidores de producción.

4. Puesta en Marcha en Desarrollo

Siga estos pasos estructurados para configurar su entorno local y comenzar el desarrollo en QUANTIFY:

Paso 1: Instalación de Dependencias

Acceda a la carpeta principal del backend e instale los paquetes de Node requeridos mediante el gestor de paquetes de npm:

```
# Navegar al directorio e instalar dependencias
cd backend
npm install
```

Paso 2: Creación de la Base de Datos Relacional

Inicie sesión en su terminal de MySQL local y ejecute la sentencia de creación para la base de datos de QUANTIFY:

```
CREATE DATABASE quantify_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Paso 3: Inicio de Servidor de Desarrollo

Inicie el servidor local en modo recarga automática para reflejar cambios al instante mediante Nodemon:

```
npm run dev
```

Salida Consola Exitosa

Si la conexión a ambas bases de datos se realiza con éxito, observará lo siguiente en su consola:

✓ Database models synchronized (MySQL).

🚀 Server running on port 5000

5. Despliegue con PM2

Para asegurar un entorno de alta disponibilidad tradicional en un VPS o servidor dedicado, se utiliza **PM2**. Este administrador de procesos permite ejecutar Node.js en clústeres, maximizando el uso de la CPU y manejando reinicios automáticos ante fallas imprevistas.

Estructura de ecosystem.config.cjs

Cree el siguiente archivo de configuración de PM2 en el directorio raíz de la API:

```
module.exports = {
  apps: [{
    name: 'quantify-backend',
    script: 'server.js',
    instances: 'max', // Crea tantos procesos como hilos de CPU tenga el servidor
    exec_mode: 'cluster', // Balanceo de carga nativo
    env_production: {
      NODE_ENV: 'production',
      PORT: 5000
    },
    max_memory_restart: '800M',
    error_file: './logs/pm2-err.log',
    out_file: './logs/pm2-out.log',
    time: true
  }]
};
```

Comandos de Control y Despliegue

Utilice la siguiente serie de comandos SSH en su servidor de producción para operar PM2:

- **Lanzamiento:** `pm2 start ecosystem.config.cjs --env production`
- **Listado de Procesos:** `pm2 list`
- **Monitoreo de Consumos:** `pm2 monit`
- **Ver logs en tiempo real:** `pm2 logs quantify-backend`
- **Persistencia tras reinicio:** `pm2 startup` seguido de `pm2 save`

6. Despliegue Containerizado

La contenedorización permite aislar la aplicación con todas sus dependencias exactas, garantizando que el software funcione de manera idéntica en cualquier servidor de producción.

Dockerfile del Backend

```
FROM node:20-alpine
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
EXPOSE 5000
CMD ["node", "--no-deprecation", "server.js"]
```

Orquestación Completa (docker-compose.yml)

```
version: '3.8'
services:
  backend:
    build: .
    ports:
      - "5000:5000"
    environment:
      - NODE_ENV=production
      - MYSQL_HOST=mysql-db
      - MONGO_URI=mongodb://mongo-db:27017/quantify_logs
    depends_on:
      - mysql-db
      - mongo-db
    restart: always
  mysql-db:
    image: mysql:8.0
    ports:
      - "3306:3306"
    environment:
```

```
MYSQL_ROOT_PASSWORD: RootPasswordSecure2026
```

```
MYSQL_DATABASE: quantify_db
```

```
volumes:
```

```
  - mysql_data:/var/lib/mysql
```

```
mongo-db:
```

```
  image: mongo:6.0
```

```
  ports:
```

```
    - "27017:27017"
```

```
  volumes:
```

```
    - mongo_data:/data/db
```

```
volumes:
```

```
  mysql_data:
```

```
  mongo_data:
```

7. Configuración de Nginx & SSL

Nginx actúa como proxy inverso frente a la API de Node.js, encargándose de gestionar las conexiones de red externas, filtrar tráfico malicioso y cifrar los datos bajo el protocolo SSL (HTTPS).

Configuración del Virtual Host

Guarde la siguiente estructura de configuración en su archivo de Nginx (habitualmente en `/etc/nginx/sites-available/quantify`):

```
server {  
    listen 80;  
    server_name api.quantifyapp.com;  
    return 301 https://$host$request_uri; # Fuerza HTTPS  
}  
  
server {  
    listen 443 ssl http2;  
    server_name api.quantifyapp.com;  
  
    ssl_certificate /etc/letsencrypt/live/api.quantifyapp.com/fullchain.pem;  
    ssl_certificate_key /etc/letsencrypt/live/api.quantifyapp.com/privkey.pem;  
  
    location / {  
        proxy_pass http://127.0.0.1:5000; # Redirige a Express  
        proxy_http_version 1.1;  
        proxy_set_header Upgrade $http_upgrade;  
        proxy_set_header Connection 'upgrade';  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
    }  
}
```

| Habilitar e Instalar Certificados SSL

Una vez creado el archivo, enlace la configuración y solicite el certificado Let's Encrypt mediante Certbot:

```
# Enlazar y verificar configuración
ln -s /etc/nginx/sites-available/quantify /etc/nginx/sites-enabled/

nginx -t

systemctl restart nginx

# Obtener certificado SSL automáticamente
certbot --nginx -d api.quantifyapp.com
```

8. Mantenimiento y Respaldos

La estabilidad post-despliegue se sostiene mediante políticas rigurosas de copias de seguridad de datos y monitoreo proactivo de la salud de los servicios.

Verificación de Salud Proactiva (Health Endpoint)

El backend de QUANTIFY expone un endpoint `/api/health`. Este responde con un estado HTTP `200` indicando la conectividad del servidor.

Se recomienda registrar este endpoint en soluciones de monitoreo automatizado (como Uptime Kuma, New Relic o Datadog) para alertar instantáneamente en caso de una caída de servicio.

Copias de Seguridad (Cron Jobs Automatizados)

Configure tareas programadas en el sistema operativo Linux mediante `crontab -e` para crear copias de seguridad de las bases de datos de MySQL y MongoDB de manera recurrente:

```
# Respalidar MySQL diariamente a las 2:00 AM (comprimido en Gzip)
0 2 * * * mysqldump -u root -p'TuPasswordSeguro' quantify_db | gzip > /opt/backups/mysql/quantify_db.sql.gz

# Respalidar MongoDB diariamente a las 2:30 AM
30 2 * * * mongodump --uri="mongodb://localhost:27017/quantify_logs" --archive=/opt/backups/mongo/quantify_logs.archive
```

Recomendación de Almacenamiento

Para cumplir con estándares internacionales de recuperación ante desastres (DRP), se aconseja programar un script adicional en bash para subir de forma encriptada los archivos resultantes `.sql.gz` y `.archive` a buckets de la nube (como AWS S3, Google Cloud Storage o Azure Blob) cada semana.

9. Resolución de Problemas

Guía rápida para diagnosticar y mitigar los errores operativos más frecuentes durante e inmediatamente después del despliegue del backend:

1. Puerto 5000 ya ocupado (EADDRINUSE)

- **Causa:** Una sesión colgada de la aplicación o un proceso ajeno está monopolizando el puerto de la API.
- **Solución en Linux:** Matar el proceso asignado en el puerto:

```
fuser -k 5000/tcp
```

- **Solución en Windows (PowerShell):** Localizar y forzar el cierre:

```
Stop-Process -Id (Get-NetTCPConnection -LocalPort 5000).OwningProcess -Force
```

2. SequelizeConnectionRefusedError (Conexión fallida a MySQL)

- **Causa:** Las credenciales de acceso son erróneas, el servidor no está corriendo, o existe un cortafuegos activo en el puerto 3306.
- **Acción correctiva:** Verifique el estado del servicio: `systemctl status mysql` en Linux. Compruebe la concordancia de contraseñas de producción definidas en el archivo `.env`.

3. Fallo de Conexión en MongoDB Atlas (Timeouts)

- **Causa:** La dirección IP pública del servidor web de producción no ha sido añadida en las políticas de seguridad de la red en el clúster de Atlas.
- **Acción correctiva:** Acceda al panel web de Atlas, navegue a **Security -> Network Access**, y añada la IP pública de su servidor actual o habilite accesos globales temporales si es necesario.